

Mitigating Dynamic Graph Distribution Shifts via Mixture of Variational Experts

Qianyu Song

qianasong@163.com

College of Electronic and Information Engineering, Shandong University of Science and Technology
Qingdao, China

Chao Li

lichao@sdust.edu.cn

College of Electronic and Information Engineering, Shandong University of Science and Technology
Qingdao, China

Yeyu Yan

23111086@bjtu.edu.cn

Institute of Information Science,
Beijing Jiaotong University
Beijing, China

Hui Zhou

zhouhui@sdust.edu.cn

College of Computer Science and Engineering, Shandong University of Science and Technology
Qingdao, China

Zhongying Zhao*

zzysuin@163.com

College of Computer Science and Engineering, Shandong University of Science and Technology
Qingdao, China

Qingtian Zeng

qtzeng@163.com

College of Computer Science and Engineering, Shandong University of Science and Technology
Qingdao, China

Abstract

Dynamic graph neural networks (DyGNNs) currently struggle with handling distribution shifts that naturally arise when training and test data follow similar but non-identical distributions. As the generation of dynamic graphs is strongly influenced by latent environments, it is critical to investigate their impacts on the generalization behavior of DyGNNs. We therefore establish a connection between the temporal message-passing scheme employed by DyGNNs and their generalization performance under distribution shifts. Our analysis reveals that environment-specific factors misguide the learning process and lead to unsatisfactory out-of-distribution (OOD) generalization. Based on this insight, we propose MoVE, a **Mixture of Variational Experts** network to mitigate complex distribution shifts in dynamic graphs. MoVE adopts a hierarchical variational architecture that extrapolates latent representations into a mixture of distribution shifts as pseudo-environments. Additionally, we incorporate a Mixture-of-Experts (MoE) framework with a novel training objective that aligns the outputs of different experts to produce invariant representations. Extensive experiments on various dynamic graphs, including both real-world and synthetic datasets, demonstrate that our model significantly outperforms state-of-the-art techniques.

CCS Concepts

- Computing methodologies → Machine learning algorithms;
- Theory of computation → Graph algorithms analysis.

Keywords

Graph Representation Learning, Dynamic Graph Neural Networks, Distribution Shifts, Out-of-Distribution Generalization

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792190>

ACM Reference Format:

Qianyu Song, Chao Li, Yeyu Yan, Hui Zhou, Zhongying Zhao, and Qingtian Zeng. 2026. Mitigating Dynamic Graph Distribution Shifts via Mixture of Variational Experts. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792190>

1 Introduction

Dynamic graphs widely exist in real-world applications, including social networks [2, 7], traffic networks [14, 28], and financial transaction networks [39], where graphs are constantly evolving over time. Recently, Dynamic Graph Neural Networks (DyGNNs) are receiving significant attention from both industry and academia due to their success in handling the complex structural and temporal information inherent in dynamic graphs.

Despite their notable success, most of existing DyGNNs are designed to optimize performance on in-distribution data, under the assumption that training and test data from dynamic graphs follow the independent and identically distributed (i.i.d.) [41]. However, recent evidence [38, 42] suggests that DyGNNs are vulnerable to out-of-distribution (OOD) data, where the underlying data-generating distributions differ from the training observations.

We illustrate this issue with a real-world example from the Reddit dataset, as shown in Figure 1(a), where user posting interactions change substantially over a one-month period due to factors such as trending topics. Such an underlying cause of the distribution shifts can be denoted as the latent environment E . In Figure 1(b), we present a causal graph structure to describe the dependencies among the dynamic graph G^t , node label Y^t under the environment E . It is broadly convinced [32, 38] that the unobserved environment acts as a common cause for both the ego-graph G^t and the labels Y^t during data generation, which leads to unsatisfactory OOD generalization. This motivates us to carefully investigate the impact of latent environment E and explore more effective graph learning strategies that better preserve graph invariance.

The consistent interactions across graph dynamics can be represented as the causal component (or *environment-insensitive* part)

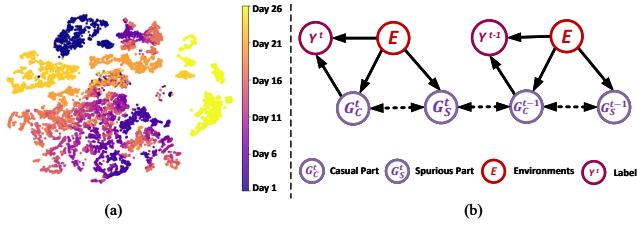


Figure 1: (a) Illustration of the temporal distribution of user posting requests in the Reddit dataset over a one-month period. (b) Investigations of the underlying cause of distribution shifts during data generation. The unobserved environment E acts as a common cause, inducing spurious correlations between $G_c^t \leftrightarrow G_s^t$ both within and across graph dynamics.

G_c^t of the ego-graph, since their relationship with the labels remains invariant across different environments [32]. In contrast, other interaction patterns exhibit considerable variation with latent environmental factors, such as interactions strongly influenced by short-lived trending topics or external events. The association between G_c^t s and the labels is therefore spurious (or *environment-sensitive*), as it is highly context-dependent and fails to hold consistently over time, making historical data less representative of current dynamics. Nevertheless, existing DyGNNs are typically trained to optimize distributional accuracy and thus exploit any available signal. As a result, the causal part in the previous time step may establish spurious correlations with the spurious part at the present time, i.e., $G_c^{t-1} \leftrightarrow G_s^t$, which is a unique challenge in dynamic scenarios. If a classifier relies heavily on such spurious patterns during training, it will inevitably fail to generalize to unseen test sets under distribution shifts.

To improve OOD generalization, the mainstream efforts have primarily focused on two lines. First, by applying data augmentation techniques to modify training data and train predictors robust to environmental shifts. For example, efforts have been made to modify graph structure [16, 17], node features [6], or graph sizes [44, 45], under the assumption that test data follow a specific type of distribution shift [18, 34]. However, the real-world distribution shifts and graph dynamics are unknown [34], which could be any combination of multiple distribution shift components [11, 22]. Fixed combinations of synthetic augmentations fail to comprehensively cover the potential shift configurations in practical scenarios [34]. The second approach attempts to disentangle input data to extract environment-invariant patterns for prediction [5, 41, 42]. However, unlike images, which can be augmented to naturally separate the main subject from the background, disentangling features in dynamic graphs is more challenging due to the complexity of topological structures (e.g., traffic networks [43], social networks [31], and recommendation systems [25]). As a result, existing approaches are generally insufficient to address this challenge effectively.

In this paper, we investigate the generalization behavior of the temporal message-passing scheme in DyGNNs and find that unobserved environments mislead them into capturing unstable relationships between ego-graph features and labels. Based on the analysis, we propose a novel and general framework named **MoVE**¹ that

handles the OOD generalization problem by capturing environment-insensitive predictive patterns to enhance generalization performance on dynamic graphs. Specifically, we integrate a variational architecture to infer a mixture of pseudo-environments as continuous latent variables by approximating the true underlying distribution of environments. To mitigate the effects of surrogate shifts, we propose a hierarchical architecture consisting of a gating model and multiple expert models, inspired by the Mixture-of-Experts (MoE) framework. Each expert model is trained to produce representations invariant within its corresponding pseudo-environment by aligning them in a shared representation space. These aligned representations are subsequently aggregated into the final representation.

Furthermore, we provide an instantiation of MoVE that operates on both continuous [8, 37] and discrete-time [23, 36] dynamic graphs and can be readily integrated with various DyGNN paradigms. Its effectiveness is validated with extensive experiments on diverse datasets covering multiple types of distribution shifts. Across these datasets, MoVE consistently achieves remarkable performance over state-of-the-art baselines. The main contributions of this work are as follows:

- **A Profound Insight:** We build a connection between the temporal message passing scheme in DyGNNs and the generalization behaviors of distribution shifts under unobserved environments. Our finding shows that environment-specific factors misguide the model toward training environment characteristics and lead to unsatisfactory OOD generalization.
- **A Novel Framework:** We propose the Mixture of Variational Experts network to mitigate complex distributional shifts in dynamic graphs. The model utilizes variational inference with a novel learning objective to infer pseudo-environments, encouraging the DyGNN encoder to focus on environment-insensitive patterns.
- **Superior Performance:** We conduct comprehensive experiments across various dynamic graphs on both real-world and synthetic datasets to evaluate the performance of the proposed method. The results demonstrate that our model significantly outperforms the state-of-the-art baselines.

2 Related works

2.1 Dynamic Graph Neural Networks

Dynamic graph neural networks are widely used to model dynamic graphs by jointly capturing spatial and temporal patterns. Existing approaches can generally be divided into two categories based on temporal granularity. The first category, Discrete-Time Dynamic Graph (DTDG) methods [21, 24], divide a dynamic graph into a sequence of snapshots and apply vanilla GNNs to each snapshot, often ignoring the temporal order of node interactions within snapshots. Sequential models such as RNNs or LSTMs are employed to capture temporal dependencies. The second category, Continuous-Time Dynamic Graph (CTDG) methods [23, 36, 37], explicitly encode the interaction times as edge features to learn time-sensitive node representations. This is accomplished using mechanisms such as temporal random walks [9, 30], graph transformers [29], or other sequential models [4] designed for continuous-time event data.

¹Code and data are available at <https://github.com/SSQiana/MoVE>.

However, most existing dynamic graph learning methods rely on the i.i.d. assumption, which limits their ability to generalize under OOD shifts in dynamic scenarios. In this paper, we analyze the fundamental challenges of OOD generalization in dynamic graph data and propose a general framework that can be applied to both CTDG and DTDG settings.

2.2 Out-of-Distribution Learning on Graphs

Out-of-distribution (OOD) generalization on graphs relies on invariant learning, which assumes the existence of invariant factors shared across diverse environments. A representative line of research is graph data augmentation [18, 33, 34], which constructs multiple training environments by perturbing graph structures to encourage the model to learn robust, environment-agnostic representations. While this approach has demonstrated effectiveness, its primary limitation lies in its reliance on predefined or rule-based perturbations, which fail to comprehensively capture the rich and complex distribution shifts observed in real-world scenarios. Another promising paradigm is disentangled representation learning [3, 35], which aims to explicitly separate features into domain-invariant (causal) and domain-specific (variant) components. This is typically achieved by identifying causal subgraphs using importance masks or by leveraging generative models [10, 38].

In the context of dynamic graphs, conventional augmentation techniques risk violating causality by inadvertently introducing future information into the current state. To address this challenge, recent advances have focused on extending disentangled frameworks to the dynamic graph setting [5, 41, 42]. Nevertheless, the presence of temporal distribution shifts in time-varying data and structural shifts in non-Euclidean data makes disentanglement significantly more challenging. Furthermore, it necessitates strong assumptions about the independence between different components within the dynamic graphs.

3 Problem Formulation

In this section, we introduce the problem definitions and notations. Throughout this paper, we use bold lowercase letters to represent vectors, and bold uppercase letters to represent matrices.

Dynamic Graph. A dynamic graph $\mathcal{G}$ over a set of nodes $\mathcal{V}$ ($|\mathcal{V}| = N$) describes the evolution of node interactions and attributes over time. It can be represented in two forms: (1) A sequence of discrete-time snapshots $\mathcal{G} = \{\mathcal{G}^t\}_{t=1}^T$, where each snapshot $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t, \mathbf{X}^t, \mathbf{A}^t)$ includes the node set $\mathcal{V}^t$, edge set $\mathcal{E}^t$, node feature matrix $\mathbf{X}^t \in \mathbb{R}^{|\mathcal{V}^t| \times d_N}$, and adjacency matrix $\mathbf{A}^t$. (2) A sequence of timestamped interactions, denoted as $\mathcal{G} = \{(u_1, v_1, t_1), (u_2, v_2, t_2), \dots\}$. In this sequence, $u_i \in \mathcal{V}$ and $v_i \in \mathcal{V}$ represent the source and destination nodes of the i -th interaction occurring at time t_i . For simplicity, we use u, v , and t to refer to a generic source node, destination node, and timestamp, respectively. Each interaction (u, v, t) is associated with a link feature vector $\mathbf{e}_{u,v}^t \in \mathbb{R}^{d_E}$. Additionally, the node features at time t are captured by the node feature matrix $\mathbf{X}^t \in \mathbb{R}^{|\mathcal{V}^t| \times d_N}$. Here, d_E and d_N denote the dimensions of the link and node features, respectively.

Dynamic Graph Learning. In dynamic graph learning, we denote by $G^{1:t}$ and Y^t the random variables representing the sequence of ego-graphs and the corresponding node labels at time t , respectively.

Their specific instances are denoted as $\mathcal{G}^{1:t}$ and y^t . The objective is to train a DyGNN $f_\theta : \mathcal{G}^{1:t} \rightarrow \hat{Y}^t$, parameterized by θ , to predict node labels in unseen test data. This work focuses on node-level prediction tasks, which are widely adopted in dynamic graph research [41, 42]. For a node v with its temporal ego-graph sequence $\mathcal{G}_v^{1:t}$, the loss function for a training instance $(\mathcal{G}_v^{1:t}, y^t)$ is defined as $\mathcal{L}(y^t, f_\theta(\mathcal{G}_v^{1:t}))$. Accordingly, the empirical risk minimization (ERM) objective can be expressed as:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathcal{G}_v^{1:t}, y^t) \sim p_{\text{tr}}(G^{1:t}, Y^t)} [\mathcal{L}(y^t, f_\theta(\mathcal{G}_v^{1:t}))], \quad (1)$$

where $p_{\text{tr}}(G^{1:t}, Y^t)$ denotes the joint training distribution over temporal graphs and labels. However, the optimization in Eqn. (1) will fail to generalize well when $p_{\text{tr}}(G^{1:t}, Y^t) \neq p_{\text{te}}(G^{1:t}, Y^t)$, where $p_{\text{tr}}/p_{\text{te}}$ refer to the training and testing distributions, respectively. Therefore, it is essential to analyze the causes of DyGNNs' generalization failure under distribution shifts and to explore potential remedies accordingly.

4 Proposed Method

In this section, we first analyze how environmental factors affect the generalization behavior of DyGNNs (Section 4.1). Building on this analysis, we then introduce the corresponding solutions within our proposed method (Section 4.2). Note that our method is a general framework applicable to both discrete-time (snapshot-based) and continuous-time dynamic graphs. For simplicity, we adopt the discrete-time dynamic graph notation in this section. Extensions of our framework to continuous-time dynamic graphs and the mixture-of-experts design are provided in Appendix B.

4.1 Generalization Analysis for DyGNNs

To understand the limitations of DyGNN generalization, we perform an analysis that uncovers how environment-specific factors shape the learning process and impair OOD performance.

Limitations of DyGNN Generalization. Existing state-of-the-art DyGNNs can be abstracted into a *generic* architecture, including *recursive temporal sampling* and *temporal message passing* [15, 36]. Specifically, for a target node v at a given time t in a dynamic graph $\mathcal{G}$, a DyGNN first samples a set of temporal neighbors $\tilde{N}_v^t$ that have previously interacted with v . Subsequently, the DyGNN aggregates features from this sampled neighborhood. These features are typically a concatenation of the neighbor's $(l-1)$ -layer embedding $\mathbf{h}_{u,t'}^{(l-1)}$, the corresponding edge feature $\mathbf{e}_{u,v}^{t'}$, and a time encoding $\tau(t-t')$. Here, the function $\tau(\cdot)$ encodes the time difference between the target timestamp t and a sampled historical interaction at time t' . The recursive aggregation of its L -hop temporal neighborhood can be expressed as:

$$\tilde{N}_v^t = \text{SAMPLE}(N_v^t), \quad (2)$$

$$\mathbf{h}_{v,t}^{(l)} = \text{AGGREGATE}(\{\mathbf{h}_{u,t'}^{(l-1)} \parallel \mathbf{e}_{u,v}^{t'} \parallel \tau(t-t') \mid (u, t') \in \tilde{N}_v^t\}), \quad \forall l = 1, \dots, L. \quad (3)$$

where $N_v^t = \{u, t' \mid t' < t\}$ denotes the set of historical interactions of v before time t . In real-world scenarios, training data are collected from a mixture of environmental patterns. The environmental variable E affects the data generation process during *recursive temporal sampling*, introducing a dependency of E on $G^{1:t}$. Meanwhile, the

temporal message passing mechanism in DyGNNs further biases the model toward environment-sensitive relationships, leading to a dependency of $G^{1:t}$ on $\hat{Y}$. Consequently, the overall learning process establishes an indirect dependency between E and the prediction results $\hat{Y}$. To formally describe this, we reformulate Eqn. (1) to characterize the model fitting process on temporal graph data $\mathcal{G}^{1:t}$ under unobserved environmental conditions:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{e \sim p_{Tr}(E)} \left[\mathbb{E}_{(\mathcal{G}^{1:t}, Y^t) \sim p_{Tr}(\mathcal{G}^{1:t}, Y^t | e)} [\mathcal{L}(Y^t, f_{\theta}(\mathcal{G}^{1:t}))] \right]. \quad (4)$$

This formulation suggests that the fundamental limitation of DyGNNs in OOD generalization arises from their reliance on environment-specific factors E , which bias the learned parameters θ^* toward the characteristics of the training environments [32].

Strategies for Enhancing OOD Generalization. Based on the preceding analysis, we identify two potential solutions to improve OOD generalization. The first is to mitigate the influence of environment specific features, thereby preventing the model from overfitting to a specific environment (i.e., neutralizing the effect of E on $G^{1:t}$). The second is to disentangle causal patterns from non-causal correlations within and across temporal dynamics (i.e., removing the dependency between G_c^{t-1} and G_s^t).

However, the second approach presents significant challenges. Disentangling features in dynamic graphs is less straightforward due to the complex nature of topological structures. This disentanglement would necessitate strong assumptions about the distributional independence between G_c^t and G_s^t during the data generation. Consequently, we adopt the strategy of training the model to mitigate the influence of the specific environment E on $G^{1:t}$ by generating multiple environments. One common strategy is to directly manipulate the observed data (e.g., through graph augmentation [18, 33, 34]). However, it typically yields only a limited number of discrete environments and fails to capture the continuous nature of dynamic OOD shifts. We instead employ a learning-based approach by inferring the latent environments based on the observed data.

4.2 Variational Inference for Distribution Extrapolation

The framework of the proposed MoVE is illustrated in Figure 2. We treat pseudo-environments as continuous latent variables and employ a variational architecture to infer them by enumerating possible environment contexts [32]. The inferred latent variables are then regularized with Kullback–Leibler (KL) divergence to align with a uniform and independent prior, such that the pseudo-environments remain independent of the ego-graph.

Latent Distribution Inference. MoVE employs a variational framework designed for dynamic graphs. Specifically, the latent node representations are obtained by inferring the posterior distribution; these representations are low-dimensional vectors that facilitate subsequent environment inference and graph invariant representation learning:

$$q(\mathbf{Z}^t | \mathbf{A}^t, \mathbf{X}^t) = \mathcal{N}(\boldsymbol{\mu}^t, \text{diag}((\boldsymbol{\sigma}^t)^2)), \quad (5)$$

where $\mathbf{X}^t$ is the feature matrix (i.e., a concatenation of node features and time encodings), $\mathbf{A}^t$ is the adjacency matrix of the graph $\mathcal{G}^t$, $\boldsymbol{\mu}^t$ and $\boldsymbol{\sigma}^t$ are the parameters of the posterior distribution. We use ϕ to denote the encoder which computes the mean and standard

deviation parameters as $\boldsymbol{\mu}^t = \phi_{\mu}(\mathbf{X}^t, \mathbf{A}^t)$ and $\boldsymbol{\sigma}^t = \phi_{\sigma}(\mathbf{X}^t, \mathbf{A}^t)$, respectively. We employ the *reparameterization trick* for gradient descent during training $\mathbf{z}^t = \boldsymbol{\mu}^t + \boldsymbol{\sigma}^t \odot \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\odot$ denotes the element-wise product.

Mixture of Posteriors as Pseudo-Environments. We introduce an *environment estimator* composed of K perturbators, denoted as $g_{\omega_k}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t)$ for $k = 1, \dots, K$. The objective of these perturbators is to generate a set of K distinct posterior distributions, $\{\mathbf{Z}_k^t\}_{k=1}^K$. A common practice for regularizing pseudo-environments is to independently constrain each latent distribution, $\{q_k(\mathbf{Z}_k^t)\}_{k=1}^K$, toward a shared standard normal prior. However, employing a standard Gaussian prior will excessively constrain the posterior distributions, limiting the expressive capacity of the latent space. Consequently, the learned representations lack diversity, which fails to generate a rich spectrum of pseudo-environments necessary for capturing continuous OOD shifts.

Our objective is for the latent representations of these pseudo-environments to be sufficiently informative for learning stable relationships, rather than attempting to explicitly model all potential test distribution shifts. To achieve this, the variational architecture is expected to learn useful patterns from a *Mixture of Posteriors*, which facilitates the discovery of stable relations from observed data for better generalization. Accordingly, the environment estimator is parameterized by ω , which is trained with a learning objective to effectively infer these pseudo-environments:

$$\boldsymbol{\mu}_k, \boldsymbol{\sigma}_k = g_{\omega_k}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t), \quad (6)$$

$$q(\mathbf{Z}_k^t | \mathbf{A}^t, \mathbf{X}^t) = \mathcal{N}(\mathbf{Z}_k^t | \boldsymbol{\mu}_k^t, \text{diag}((\boldsymbol{\sigma}_k^t)^2)), \quad (7)$$

where $\boldsymbol{\mu}_k^t$ and $\boldsymbol{\sigma}_k^t$ denote the parameters of the approximated posterior. To specify $g_{\omega_k}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t)$, we encourage diversity in the generated distribution shifts by maximizing their variance:

$$\mathcal{L}_{\text{shift}} = \underbrace{e^{-\gamma_{\mu} \cdot \text{Var}(\{\boldsymbol{\mu}_k^t\}_{k=1}^K)} + e^{-\gamma_{\sigma} \cdot \text{Var}(\{\boldsymbol{\sigma}_k^t\}_{k=1}^K)}}_{\text{Extrapolation}} + \underbrace{\mathbb{E}_k [\|\boldsymbol{\mu}_k^t\|_2^2 + \|\boldsymbol{\sigma}_k^t\|_2^2]}_{\text{Regularization}}. \quad (8)$$

where $\text{Var}(\cdot)$ denotes the variance across the set of conditional posterior parameters, while γ_{μ} and γ_{σ} are hyperparameters that control this diversity. The extrapolation term promotes diversity among the pseudo-environments, whereas the regularization term penalizes the parameter norms to prevent divergence.

4.3 Mixture of Variational Experts

The latent variables $\{\mathbf{Z}_k^t\}_{k=1}^K$ are sampled from distinct Gaussian distributions. To process these samples corresponding to pseudo-environments, we introduce a mixture-of-experts architecture [13]. The MoE consists of multiple experts and a gating model, where each expert serves as a specific decoder to handle a corresponding shift component. The outputs of these experts are combined to generate invariant representations under distributional shifts.

Expert models. We construct $K + 1$ expert models, denoted as $\{\xi_k\}_{k=0}^K$. Each expert model essentially serves as a decoder, transforming the latent sample $\mathbf{Z}_k^t$ into node representations $\xi_k(\mathbf{Z}_k^t)$ that capture information relevant to an individual shift component. Among these, a special *in-distribution* expert, ξ_0 , is designated as the reference model. This expert operates on the original latent

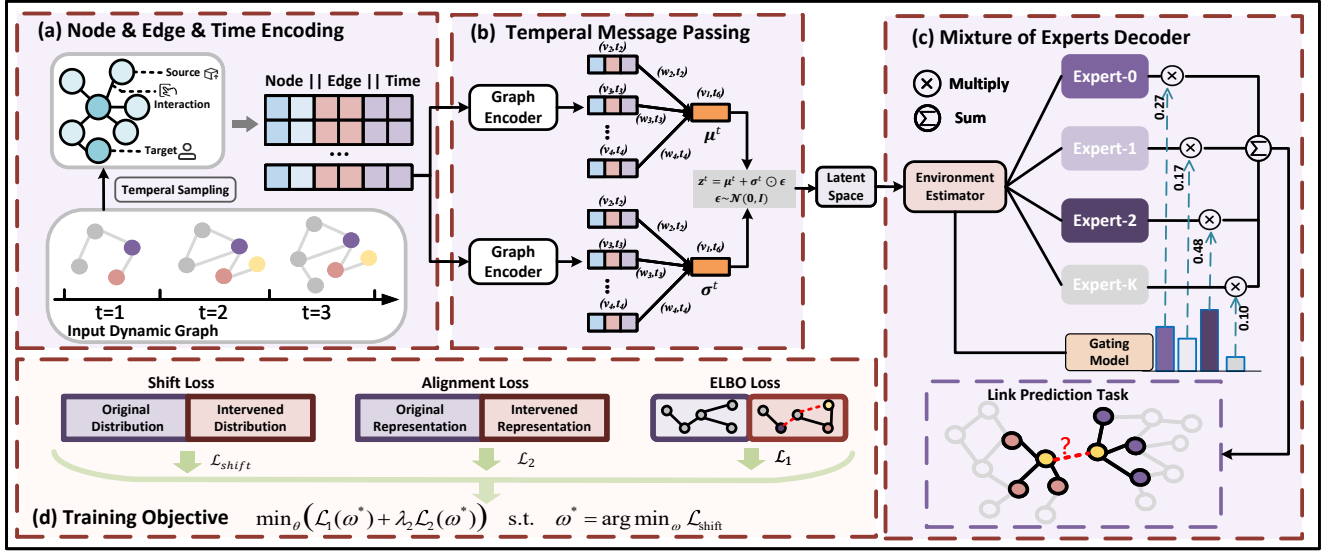


Figure 2: A conceptual diagram of the MoVE architecture. It first extracts a series of ego-graphs over time using recursive temporal sampling (a). Temporal message passing (b) is then performed to propagate information within these graphs, enabling the model to capture the evolving dependencies. We further decompose the generalization problem into two distinct phases: (c) decomposing latent representations into a mixture of pseudo-environments to account for target distribution shifts; and (d) aligning expert representations to mitigate spurious variations across environments.

representation, and its space functions as an anchor for aligning the representations of the other experts. Each of the remaining K experts, $\{\xi_k\}_{k=1}^K$, is designed to handle a specific shift component. More precisely, each remaining expert ξ_k is tasked with producing invariant representations with respect to the pseudo-environment inferred by its corresponding environment estimator, g_{ω_k} . This structure allows the reference model to provide a stable baseline, while the remaining experts specialize in discovering invariant relations under the diverse distribution shifts.

Gating model. The gating model ψ takes latent representations $\mathbf{Z}_k^t$ as input and outputs a weight vector λ on the shift components. It operates on a global context vector, formed by concatenating all $K + 1$ expert representations, to adaptively assign mixture weights based on their relative utility.

The MoE workflow. Given a mixture of $\{\mathbf{Z}_k^t\}_{k=1}^K$ latent variables, the gating model ψ assigns weights λ over the expert models. The final representation is computed via aggregating the representations based on the weight vector:

$$\mathbf{H}^t = \sum_{k=0}^K \lambda_k \xi_k(\mathbf{Z}_k^t), \quad (9)$$

$$\lambda = \text{Softmax}(\psi(\mathbf{Z}_0^t, \mathbf{Z}_1^t, \dots, \mathbf{Z}_K^t)), \quad (10)$$

Moreover, reconstructing the adjacency matrix will be conditioned on $\mathbf{H}^t$ by the classifier c :

$$p(\mathbf{A}^t | \mathbf{H}^t) = \prod_{u=1}^N \prod_{v=1}^N \text{Sigmoid}(c(\mathbf{H}_u^t, \mathbf{H}_v^t)). \quad (11)$$

4.4 Training Objective

- **ELBO Loss:** $\mathcal{L}_1$ The Evidence Lower Bound (ELBO) loss is derived from the variational lower bound (see derivation in Appendix A). More precisely, the evidence lower bound can be written as follows:

$$\mathcal{L}_1 = \sum_{t=1}^T \left[\mathbb{E}_{\mathbf{Z}_0^t \sim q_0 \dots \mathbf{Z}_K^t \sim q_K} [\log p(\mathbf{A}^t | \mathbf{H}^t)] - \frac{1}{K+1} \sum_{k=0}^K \text{D}_{\text{KL}}(q_k(\mathbf{Z}_k^t | \mathbf{A}^t, \mathbf{X}^t) \parallel p(\mathbf{Z}_k^t)) \right]. \quad (12)$$

- **Alignment Loss $\mathcal{L}_2$:** Constraining the individual posteriors $q_k(\mathbf{Z}_k^t)$ with ELBO loss alone may not be sufficient, since the MoE in Eqn. (9) aggregates representations from all $K + 1$ independently learned posteriors. Aggregating these independently learned representations without proper alignment leads to inconsistencies during model training. We can explain it from two perspectives: (i) Perturbations in the latent space can lead the shifted distribution $\{\mathbf{Z}_k^t\}_{k=0}^K$ to exhibit high variance, making it difficult for the predictor to generate stable predictions. (ii) The reference model ξ_0 ($k = 0$) serves as an in-distribution expert. Without alignment, node representations cannot remain invariant to controlled distribution shifts, resulting in unstable predictions under extrapolated scenarios. To achieve this, for each perturbed expert ξ_k , we minimize the expected distance between its output $\xi_k(\mathbf{Z}_k^t)$ and the reference expert's output $\xi_0(\mathbf{Z}_0^t)$:

$$\mathcal{L}_2 = \frac{1}{K} \sum_{t=1}^T \sum_{k=1}^K \mathbb{E}_{\mathbf{Z}_0^t \sim q_0, \mathbf{Z}_k^t \sim q_k} [\|\xi_k(\mathbf{Z}_k^t) - \xi_0(\mathbf{Z}_0^t)\|^2]. \quad (13)$$

• **Total Loss:** We consider collaborative learning of two components: (i) the environment estimators $\{g_{\omega_k}\}_{k=1}^K$ with parameter ω , which generate latent distributions as inferred pseudo-environments, and (ii) the main model with parameter θ , consisting of the encoder ϕ , the expert decoders $\{\xi_k\}_{k=0}^K$, and the gating model ψ , which capture the stable correlation between ego-graphs and labels. The model is practically solved using alternating gradient steps for ω and θ , Alg. 1 in the appendix presents the model's feed-forward and training. The overall training objective is given as:

$$\min_{\theta} \left(\mathcal{L}_1(\omega^*) + \alpha \cdot \mathcal{L}_2(\omega^*) \right) \quad \text{s.t.} \quad \omega^* = \arg \min_{\omega} \mathcal{L}_{\text{shift}}, \quad (14)$$

where α balances the influence of alignment loss $\mathcal{L}_2$.

5 Experiments

In this section, we evaluate MoVE on both ID and OOD settings of dynamic graphs, respectively corresponding to in-distribution and out-of-distribution scenarios, covering continuous-time and discrete-time cases.

5.1 Experiment Setup

Baselines. We conduct a comprehensive evaluation of MoVE against Empirical Risk Minimization (ERM) and two major categories of baseline methods. The first category includes standard DyGNNs developed for different temporal settings. For continuous-time dynamic graphs, we consider TGAT [36], TGN [23], GraphMixer [4], DygFormer [37], and FreeDyG [27]. For discrete-time snapshot graphs, we evaluate VGAE [10], EGCN [21], and DySAT [24]. The second category comprises methods for OOD generalization, including invariant learning approaches such as IRM [1], and CaNet [32], and methods specifically for dynamic graph OOD, namely DIDA [41], I-DIDA [40], SILD [42], and EAGLE [38]. Detailed descriptions of all baselines and implementation details are provided in the Appendix B.3.

Training Settings. For discrete-time snapshots, our approach processes graph snapshots sequentially, leveraging information from the current timestep t to predict edges at the subsequent timestep $t + 1$. In this setting, we employ GCN as the encoder backbone to compute node representations. For continuous-time graphs, we utilize the DyGLib [37] framework to ensure a standardized training pipeline across all methods. A Transformer-based architecture serves as the encoder backbone for computing node representations in this context. For all datasets, we adopt the inductive link prediction task, which requires the model to utilize historical graph snapshots to predict emerging links in the subsequent time step [37, 42]. For all datasets, we conduct five independent runs and report the mean and standard deviation of the results. Additional implementation details and baseline descriptions are provided in Appendix B.

5.2 Comparative Results

5.2.1 Distribution Shifts on Discrete Time Dynamic Graphs. We evaluate MoVE on three real-world discrete-time dynamic graph datasets: COLLAB, YELP, and ACT. We adopt the challenging inductive link prediction task, where the model leverages historical graphs to predict link occurrences at the next time step. To further measure the model's performance under distribution shifts, the

model is trained on a source graph and evaluated on a target graph from a distinct domain that remains unseen during training. More details on their distribution shifts are provided in Appendix B.2.

Results and Observations. Table 1 presents the primary results, distinguishing between test sets with OOD shifts and in-distribution (ID) settings. We have the following observations: (i) While the baseline models perform well on the training data, their effectiveness consistently declines during testing. This finding supports the hypothesis that DyGNNs are prone to overfitting spurious, non-causal features, which hinders their generalization. (ii) MoVE consistently outperforms all baselines under these distribution shifts. We attribute this enhanced performance to our training objective, which promotes invariance to changes across pseudo-environments, thereby mitigating the influence of environment-specific features.

5.2.2 Distribution Shifts on Continuous Time Dynamic Graphs. We further evaluate MoVE on four real-world continuous-time dynamic graph datasets: Amazon-Review, Reddit, UN Trade, and UN Vote. In Appendix B.2, we illustrate the distribution shifts in these datasets with respect to the average neighbor degree. Table 2 presents the results under different data splits. MoVE achieves overall superior performance across datasets, with particularly notable gains on the UN Trade dataset. This demonstrates the effectiveness of MoVE in addressing OOD generalization over continuous-time dynamic graphs under inductive learning.

5.2.3 Distribution Shifts on Synthetic Dynamic Graphs. We follow [42] to synthesize additional node features that simulate distribution shifts on the COLLAB dataset. At each time step, we sample a proportion $p(t)$ of positive links and $(1 - p(t))$ of negative links from the next-step edges A^{t+1} , where $p(t) = \bar{p} + \sigma \cos(t)$. The original and synthesized features are then concatenated as $[X^t \parallel X^{t'}]$ and fed into the model. Specifically, $X^{t'}$ is obtained by training embeddings to minimize the reconstruction loss $\ell(X^{t'} X^{t'}{}^T, A^{t+1})$, where $\ell(\cdot)$ denotes the cross-entropy loss function [38]. A larger $p(t)$ induces stronger spurious correlations between $X^{t'}$ and future environments, causing models that rely on such correlations to generalize poorly. To control this effect, we vary $\bar{p}$ during training (0.4, 0.6, 0.8) and fix it to 0.1 during testing. The results are shown in Table 4, which demonstrate that MoVE effectively captures invariant patterns across different levels of distribution shift. As the shift intensity increases, almost all baselines experience substantial performance degradation, whereas the performance of MoVE declines only marginally. This indicates that MoVE is more robust in eliminating spurious correlations under severe OOD environments.

5.3 Ablation Studies

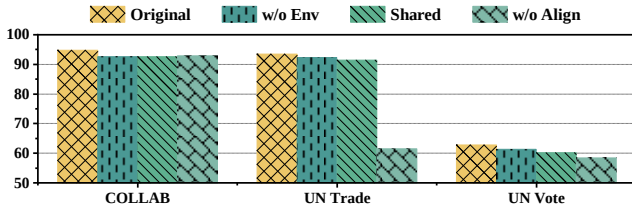
We conduct an ablation study to evaluate the effectiveness of the key components in MoVE by systematically removing them. The results are presented in Figure 3. We analyze three primary variants of our model: w/o Env, which removes the environment shift loss ($\mathcal{L}_{\text{shift}}$) and therefore no longer promotes diversity among pseudo-environments; Shared, which replaces the Mixture-of-Experts architecture with a single shared encoder; and w/o Align, which is trained without the representation alignment loss ($\mathcal{L}_2$). Additionally, we investigate the effects of the Regularization term in Eqn. (8) and the Alignment term in Eqn. (13), as shown in Figure 4.

Table 1: Experimental results on discrete time dynamic graphs for link prediction task (AP%).

Dataset		COLLAB		Yelp		ACT	
Spilt		ID	OOD	ID	OOD	ID	OOD
Base	ERM	86.34 $\pm$ 0.00	84.36 $\pm$ 0.00	62.39 $\pm$ 0.00	61.37 $\pm$ 0.00	91.78 $\pm$ 0.00	89.31 $\pm$ 0.00
Invariant Learning	IRM	83.16 $\pm$ 0.46	82.27 $\pm$ 0.74	68.95 $\pm$ 0.00	63.95 $\pm$ 0.00	<u>92.09 $\pm$ 0.03</u>	<u>90.16 $\pm$ 0.05</u>
	CaNet	81.56 $\pm$ 0.75	81.29 $\pm$ 0.47	61.34 $\pm$ 0.46	51.41 $\pm$ 0.74	83.14 $\pm$ 0.42	84.42 $\pm$ 0.74
Dynamic Graph Learning	VGAE	88.74 $\pm$ 0.04	<u>86.98 $\pm$ 0.01</u>	66.29 $\pm$ 0.00	62.24 $\pm$ 0.00	89.93 $\pm$ 0.00	89.51 $\pm$ 0.00
	EGCN	83.38 $\pm$ 0.69	77.29 $\pm$ 0.61	71.36 $\pm$ 0.29	65.52 $\pm$ 1.64	71.65 $\pm$ 0.27	65.54 $\pm$ 0.17
	DySAT	80.24 $\pm$ 0.00	77.57 $\pm$ 0.00	43.00 $\pm$ 0.00	46.73 $\pm$ 0.00	72.58 $\pm$ 0.00	63.58 $\pm$ 0.00
Dynamic Graph OOD	DIDA	<u>90.27 $\pm$ 0.48</u>	81.21 $\pm$ 0.37	76.53 $\pm$ 2.03	74.23 $\pm$ 2.23	83.61 $\pm$ 0.73	77.07 $\pm$ 0.45
	I-DIDA	89.77 $\pm$ 0.48	82.21 $\pm$ 0.37	72.53 $\pm$ 0.52	70.65 $\pm$ 0.64	85.32 $\pm$ 0.00	85.23 $\pm$ 0.00
	SILD	79.51 $\pm$ 0.00	77.27 $\pm$ 0.06	69.23 $\pm$ 0.04	64.89 $\pm$ 0.00	82.72 $\pm$ 0.00	76.48 $\pm$ 0.00
	EAGLE	84.18 $\pm$ 0.37	82.68 $\pm$ 0.28	68.39 $\pm$ 0.28	64.39 $\pm$ 0.34	84.38 $\pm$ 0.18	82.72 $\pm$ 0.38
Ours		90.80 $\pm$ 0.00	89.13 $\pm$ 0.02	<u>74.12 $\pm$ 0.06</u>	<u>72.13 $\pm$ 0.08</u>	94.85 $\pm$ 0.01	94.41 $\pm$ 0.00

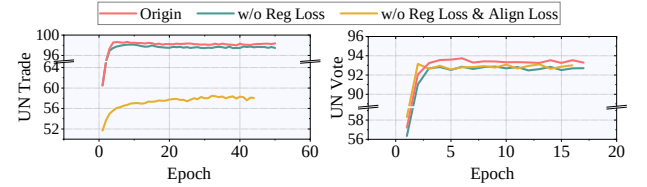
Table 2: Experimental results on continuous time dynamic graphs for link prediction task (AP%).

Dataset		Amazon-Review		Reddit		UN Trade		UN Vote	
Spilt		Train	Test	Train	Test	Train	Test	Train	Test
Base	ERM	83.09 $\pm$ 0.14	67.98 $\pm$ 0.72	92.49 $\pm$ 0.21	77.29 $\pm$ 0.94	62.49 $\pm$ 0.25	61.06 $\pm$ 0.31	58.14 $\pm$ 0.36	51.88 $\pm$ 0.95
Invariant Learning	IRM	82.39 $\pm$ 0.30	68.16 $\pm$ 0.62	92.19 $\pm$ 0.71	76.33 $\pm$ 4.06	62.21 $\pm$ 0.79	60.82 $\pm$ 1.04	57.11 $\pm$ 0.24	50.22 $\pm$ 1.02
	CaNet	84.20 $\pm$ 0.28	71.26 $\pm$ 0.40	94.42 $\pm$ 0.53	81.53 $\pm$ 0.86	<u>73.25 $\pm$ 0.39</u>	<u>68.37 $\pm$ 1.26</u>	59.17 $\pm$ 0.36	51.26 $\pm$ 0.74
Dynamic Graph Learning	TGAT	78.14 $\pm$ 0.43	63.09 $\pm$ 0.82	90.10 $\pm$ 0.42	74.68 $\pm$ 0.69	55.71 $\pm$ 0.24	62.21 $\pm$ 0.32	57.19 $\pm$ 0.84	51.57 $\pm$ 1.83
	TGN	75.35 $\pm$ 0.69	71.42 $\pm$ 0.37	95.34 $\pm$ 0.19	83.41 $\pm$ 0.42	66.17 $\pm$ 0.79	62.33 $\pm$ 2.31	<u>76.19 $\pm$ 1.28</u>	<u>57.41 $\pm$ 3.10</u>
	GraphMixer	<u>88.09 $\pm$ 0.29</u>	<u>86.35 $\pm$ 0.25</u>	<u>98.18 $\pm$ 0.36</u>	<u>95.13 $\pm$ 0.58</u>	64.36 $\pm$ 0.46	59.97 $\pm$ 0.14	58.21 $\pm$ 0.57	51.22 $\pm$ 0.75
	DyGFormer	67.28 $\pm$ 0.15	67.36 $\pm$ 0.10	95.64 $\pm$ 0.20	88.09 $\pm$ 0.64	67.19 $\pm$ 0.59	63.31 $\pm$ 0.46	60.20 $\pm$ 0.27	52.22 $\pm$ 0.41
	FreeDyg	87.64 $\pm$ 0.79	74.13 $\pm$ 0.14	97.43 $\pm$ 0.71	94.84 $\pm$ 0.90	67.22 $\pm$ 0.51	66.45 $\pm$ 0.15	59.07 $\pm$ 0.18	54.42 $\pm$ 0.21
Dynamic Graph OOD	DIDA	83.13 $\pm$ 0.27	69.32 $\pm$ 0.25	93.71 $\pm$ 0.15	74.05 $\pm$ 0.79	57.63 $\pm$ 0.15	52.38 $\pm$ 0.00	58.06 $\pm$ 0.61	53.34 $\pm$ 0.27
	SILD	82.60 $\pm$ 0.08	67.94 $\pm$ 0.10	92.21 $\pm$ 0.23	75.50 $\pm$ 0.11	56.33 $\pm$ 0.33	52.77 $\pm$ 0.24	58.10 $\pm$ 0.32	51.67 $\pm$ 0.36
Ours		98.75 $\pm$ 0.24	95.46 $\pm$ 0.24	98.23 $\pm$ 0.53	96.61 $\pm$ 0.24	96.35 $\pm$ 0.65	94.42 $\pm$ 0.24	91.97 $\pm$ 0.42	58.26 $\pm$ 0.10

**Figure 3: Ablation studies.**

• **Ablation Study on the Environment Estimator.** Removing the environment inference module (w/o Env) leads to a performance degradation. This result validates the effectiveness of our approach, confirming that the estimator is crucial for capturing sufficient information to learn stable, causal relationships.

• **Ablation Study on the Mixture of Experts.** Using a single shared encoder (Shared) instead of specialized experts also results in a notable performance decrease. We attribute this to the limited

**Figure 4: Learning curves on the UN Trade and UN Vote datasets with/without alignment and regularization losses.**

expressive capacity of a single expert. This limitation hinders alignment with the reference model and reduces the model’s ability to remain invariant to specific distributional shifts.

• **Ablation Study on the Regularization and Alignment Design.** To investigate the individual contributions of the proposed regularization (Eqn. (8)) and alignment (Eqn. (13)) terms, we conduct an additional ablation study. As shown in Figure 3, our primary observation is that removing the alignment term results in a substantial performance drop, particularly on the UN Trade and UN

Vote datasets. To further examine the role of regularization, we compare the learning curves of average precision under two conditions: (1) removing only the regularization loss from Eqn. (8), and (2) removing both the regularization and alignment terms, as presented in Figure (4). The results indicate that ablating only the regularization term causes a slight decline in average precision, whereas removing both terms leads to a severe performance collapse on the UN Trade dataset. This suggests that without the alignment, the expert models diverge and learn distinct, incompatible representation spaces. The MoE architecture then struggles to process these mixed representations. Therefore, the alignment loss plays a critical role in preserving a coherent representation space and improving generalization performance.

5.4 Hyper-parameter Analysis

In this section, we investigate the sensitivity of our model to key hyperparameters. We specifically analyze the impact of the number of experts K , the alignment weight α , and the diversity regularizers γ_μ and γ_σ .

Impact of parameter K . The parameter K determines the number of pseudo-environments generated by the environment estimator, which corresponds to the number of specialized experts in the MoE architecture. We investigate its impact by varying its value, with the results presented in Figure 5. The findings indicate a consistent trend across different datasets: as K increases, the model's performance generally improves. This suggests that generating a richer variety of pseudo-environments allows the model to explore a wider spectrum of potential data distributions.

Impact of parameter α . The hyperparameter α regulates the alignment loss $\mathcal{L}_2$ between the perturbed experts and the reference expert ξ_0 . The results are presented in Figure 5. On the COLLAB dataset, the decline in performance with a larger α suggests that a strong alignment constraint suppresses the diversity of pseudo-environments required for generalization. Conversely, on the UN Trade and UN Vote datasets, the performance improvement with a larger α indicates that the constraint serves as an effective regularizer, enforcing consistency across experts.

Impact of parameter γ_μ and γ_σ . The parameters γ_μ and γ_σ regulate the diversity of pseudo-environments by encouraging variance in the means and standard deviations of the posterior distributions in Eqn. (8). As illustrated in Figure 7, a larger γ_σ proves beneficial, as it enables the generation of pseudo-environments with relatively stable means but high variability in their uncertainties. This richer exploration of variance facilitates the discovery of invariant patterns across environments.

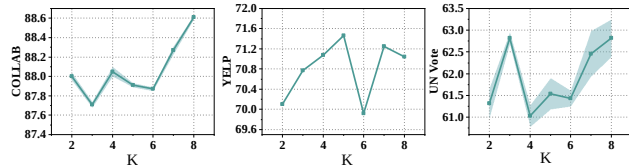


Figure 5: Sensitivity analysis of the hyperparameter K .

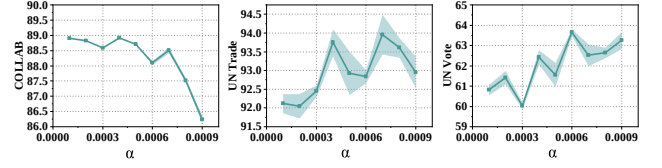


Figure 6: Sensitivity analysis of the hyperparameter α .

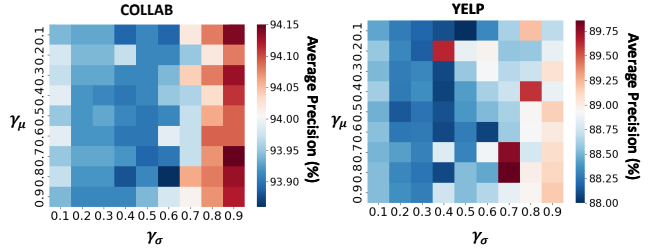


Figure 7: Parameter sensitivity analysis of γ_μ and γ_σ .

5.5 Patterns of Environment Estimators

We visualize the weight matrices of the environment estimators $\{g_{\omega_k}\}_{k=1}^K$ in Figures 9, 10, and 11. As illustrated, the learned weight matrices of the three estimators exhibit markedly different patterns. These visual distinctions indicate that the estimators have specialized in capturing unique and complementary perturbation functions. Such diversity is crucial for synthesizing a rich spectrum of pseudo-environments, and by aligning on these extrapolated distributions, the MoE architecture is encouraged to learn invariant and generalizable representations.

6 Conclusion

In this paper, we propose a novel and general framework MoVE to address the challenge of improving the generalization of DyGNNs to real-world data splits and dynamic graph distributions. MoVE utilizes a variational architecture to extrapolate observed data by generating a mixture of pseudo-environments in the latent space. A corresponding Mixture of Variational Experts model then learns to produce environment-insensitive representations from these synthesized shifts, which are aggregated to form a final prediction robust to distributional changes. The proposed method can be paired with various DyGNN frameworks, and the extensive experiments demonstrate that MoVE consistently outperforms state-of-the-art baselines across diverse datasets. A primary limitation lies in the implicit interpretability of the learned pseudo-environments, whose semantics remain implicit, we leave the development of more explainable representations for further exploration.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant No. 62472263, 52374221), the Natural Science Foundation of Shandong Province (Grant No. ZR2024MF034), Humanities and Social Science Research Project of the Ministry of Education, China (Grant No. 24YJAZH058), the CAHE 2024 Higher Education Scientific Research Planning Project (Grant No. 24XX0101).

References

- [1] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. 2019. Invariant Risk Minimization. *arXiv e-prints* (2019), arXiv:1907.
- [2] Tanya Y. Berger-Wolf and Jared Saia. 2006. A framework for analysis of dynamic social networks (*KDD '06*). Association for Computing Machinery, New York, NY, USA, 523–528.
- [3] Ruichu Cai, Zijian Li, Pengfei Wei, Jie Qiao, Kun Zhang, and Zhifeng Hao. 2019. Learning disentangled semantic representation for domain adaptation. In *IJCAI: proceedings of the conference*, Vol. 2019. 2060.
- [4] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures For Temporal Networks?. In *The Eleventh International Conference on Learning Representations*.
- [5] Lanting Fang, Yulian Yang, Kai Wang, Shanshan Feng, Kaiyu Feng, Jie Gui, Shuliang Wang, and Yew-Soon Ong. 2024. SIG: Efficient Self-Interpretable Graph Neural Network for Continuous-time Dynamic Graphs. *arXiv preprint arXiv:2405.19062* (2024).
- [6] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural networks for semi-supervised learning on graphs. *Advances in neural information processing systems* 33 (2020), 22092–22103.
- [7] Derek Greene, Donal Doyle, and Padraig Cunningham. 2010. Tracking the evolution of communities in dynamic social networks. In *2010 international conference on advances in social networks analysis and mining*. IEEE, 176–183.
- [8] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2023. Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems* 36 (2023), 2056–2073.
- [9] Ming Jin, Yuan-Fang Li, and Shirui Pan. 2022. Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs. *Advances in Neural Information Processing Systems* 35 (2022), 19874–19886.
- [10] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [11] Boris Knyazev, Graham W Taylor, and Mohamed Amer. 2019. Understanding attention and generalization in graph neural networks. *Advances in neural information processing systems* 32 (2019).
- [12] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.
- [13] Bo Li, Yifei Shen, Jingkang Yang, Yezhen Wang, Jiawei Ren, Tong Che, Jun Zhang, and Ziwei Liu. 2023. Sparse Mixture-of-Experts are Domain Generalizable Learners. In *The Eleventh International Conference on Learning Representations*.
- [14] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Fan Yang, Funing Sun, Depeng Jin, and Yong Li. 2023. Dynamic Graph Convolutional Recurrent Network for Traffic Prediction: Benchmark and Solution. *ACM Transactions on Knowledge Discovery from Data* 17, 1, Article 9 (2023), 21 pages.
- [15] Yiming Li, Yanyan Shen, Lei Chen, and Mingxuan Yuan. 2023. Zebra: When temporal graph neural networks meet temporal personalized pagerank. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1332–1345.
- [16] Gang Liu, Tong Zhao, Jiaxin Xu, Tengfei Luo, and Meng Jiang. 2022. Graph rationalization with environment-based augmentations. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1069–1078.
- [17] Songtao Liu, Rex Ying, Hanze Dong, Lanqing Li, Tingyang Xu, Yu Rong, Peilin Zhao, Junzhou Huang, and Dinghao Wu. 2022. Local augmentation for graph neural networks. In *International conference on machine learning*. PMLR, 14054–14072.
- [18] Yang Liu, Xiang Ao, Fuli Feng, Yunshan Ma, Kuan Li, Tat-Seng Chua, and Qing He. 2023. FLOOD: A flexible invariant learning framework for out-of-distribution generalization on graphs. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 1548–1558.
- [19] Amirhossein Nadiri and Frank W Takes. 2022. A large-scale temporal analysis of user lifespan durability on the Reddit social media platform. In *Companion Proceedings of the Web Conference 2022*. 677–685.
- [20] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 188–197.
- [21] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*.
- [22] Leto Peel, Jean-Charles Delvenne, and Renaud Lambiotte. 2018. Multiscale mixing patterns in networks. *Proceedings of the National Academy of Sciences* 115, 16 (2018), 4057–4062.
- [23] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML 2020 Workshop on Graph Representation Learning*.
- [24] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. 2020. DySAT: Deep Neural Representation Learning on Dynamic Graphs via Self-Attention Networks. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 519–527.
- [25] Yatong Sun, Bin Wang, Zhu Sun, Xiaochun Yang, and Yan Wang. 2023. Theoretically guaranteed bidirectional data rectification for robust sequential recommendation. *Advances in Neural Information Processing Systems* 36 (2023), 2850–2876.
- [26] Jie Tang, Sen Wu, Jimeng Sun, and Hang Su. 2012. Cross-domain collaboration recommendation. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1285–1293.
- [27] Yuxing Tian, Yiyang Qi, and Fan Guo. 2024. Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction. In *The Twelfth International Conference on Learning Representations*.
- [28] Patara Trirat and Jae-Gil Lee. 2021. DF-TAR: A Deep Fusion Network for Citywide Traffic Accident Risk Prediction with Dangerous Driving Behavior. In *Proceedings of the Web Conference 2021*. Association for Computing Machinery, 1146–1156.
- [29] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. 2021. TCL: Transformer-based Dynamic Graph Modelling via Contrastive Learning. *arXiv e-prints* (2021), arXiv:2105.
- [30] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *International Conference on Learning Representations*.
- [31] Yanbang Wang and Jon Kleinberg. 2023. On the relationship between relevance and conflict in online social link recommendations. *Advances in Neural Information Processing Systems* 36 (2023), 36708–36725.
- [32] Qitian Wu, Fan Nie, Chenxiao Yang, Tianyi Bao, and Junchi Yan. 2024. Graph Out-of-Distribution Generalization via Causal Intervention. In *Proceedings of the ACM Web Conference 2024*. Association for Computing Machinery, 850–860.
- [33] Qitian Wu, Hengrui Zhang, Junchi Yan, and David Wipf. 2022. Handling Distribution Shifts on Graphs: An Invariance Perspective. In *International Conference on Learning Representations*.
- [34] Shirley Wu, Kaidi Cao, Bruno Ribeiro, James Y Zou, and Jure Leskovec. 2024. GraphMETRO: Mitigating Complex Graph Distribution Shifts via Mixture of Aligned Experts. *Advances in Neural Information Processing Systems* 37 (2024), 9358–9387.
- [35] Yingxin Wu, Xiang Wang, An Zhang, Xiangnan He, and Tat-Seng Chua. 2022. Discovering Invariant Rationales for Graph Neural Networks. In *International Conference on Learning Representations*.
- [36] Da Xu, chuanwei ruan, evren korpeoglu, sushant kumar, and kannan achan. 2020. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations*.
- [37] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems* 36 (2023), 67686–67700.
- [38] Haonan Yuan, Qingyun Sun, Xingcheng Fu, Ziwei Zhang, Cheng Ji, Hao Peng, and Jianxin Li. 2023. Environment-Aware Dynamic Graph Learning for Out-of-Distribution Generalization. In *Advances in Neural Information Processing Systems*, Vol. 36. 49715–49747.
- [39] Shilei Zhang, Toyotaro Suzumura, and Li Zhang. 2021. DynGraphTrans: Dynamic Graph Embedding via Modified Universal Transformer Networks for Financial Transaction Data. In *2021 IEEE International Conference on Smart Data Services (SMDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 184–191.
- [40] Zeyang Zhang, Xin Wang, Haibo Chen, Haoyang Li, and Wenwu Zhu. 2024. Disentangled Dynamic Graph Attention Network for Out-of-Distribution Sequential Recommendation. *ACM Trans. Inf. Syst.* 43, 1, Article 19 (Dec. 2024), 42 pages.
- [41] Zeyang Zhang, Xin Wang, Ziwei Zhang, Haoyang Li, Zhou Qin, and Wenwu Zhu. 2022. Dynamic graph neural networks under spatio-temporal distribution shift. *Advances in neural information processing systems* 35 (2022), 6074–6089.
- [42] Zeyang Zhang, Xin Wang, Ziwei Zhang, Zhou Qin, Weigao Wen, Hui Xue, Haoyang Li, and Wenwu Zhu. 2023. Spectral invariant learning for dynamic graphs under distribution shifts. *Advances in Neural Information Processing Systems* 36 (2023), 6619–6633.
- [43] Ziming Zhao, Zhaoxuan Li, Zhuoxue Song, Wenhao Li, and Fan Zhang. 2024. Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection. In *Proceedings of the ACM Web Conference 2024*. 1608–1619.
- [44] Yangze Zhou, Gitta Kutyniok, and Bruno Ribeiro. 2022. OOD link prediction generalization capabilities of message-passing GNNs in larger test graphs. *Advances in Neural Information Processing Systems* 35 (2022), 20257–20272.
- [45] Qi Zhu, Natalia Ponomareva, Jiawei Han, and Bryan Perozzi. 2021. Shift-robust gnn: Overcoming the limitations of localized graph training data. *Advances in Neural Information Processing Systems* 34 (2021), 27965–27977.

A Derivation for the ELBO Loss ($\mathcal{L}_1$)

The training objective $\mathcal{L}_1$ is the variational lower bound on the log-likelihood of the observed graph adjacency matrix $\mathbf{A}^t$ at each timestep t . The goal is to maximize this lower bound, which serves as a tractable proxy for the true log-likelihood $\log P(\mathbf{A}^t | \mathbf{X}^t)$. For clarity, we omit the conditioning on $\mathbf{X}^t$ in the notation below. Our objective is to maximize the log-likelihood of the adjacency matrix, which can be expressed by marginalizing over all possible values of the discrete latent variables $\{\mathbf{Z}_k^t\}_{k=0}^K$:

$$\log P(\mathbf{A}^t) = \log \sum_{\{\mathbf{Z}_k^t\}_{k=0}^K} P(\mathbf{A}^t, \mathbf{Z}_0^t, \dots, \mathbf{Z}_K^t). \quad (15)$$

Since this summation is intractable, we introduce a joint variational posterior distribution $Q(\{\mathbf{Z}_k^t\}_{k=0}^K | \mathbf{A}^t, \mathbf{X}^t)$, which approximates the true posterior $P(\{\mathbf{Z}_k^t\}_{k=0}^K | \mathbf{A}^t)$. We assume this joint distribution factorizes over the individual pseudo-environments:

$$Q(\{\mathbf{Z}_k^t\}_{k=0}^K | \mathbf{A}^t, \mathbf{X}^t) = \prod_{k=0}^K q_k(\mathbf{Z}_k^t | \mathbf{A}^t, \mathbf{X}^t). \quad (16)$$

We rewrite the log-likelihood by introducing the variational posterior inside the summation:

$$\log P(\mathbf{A}^t) = \log \sum_{\{\mathbf{Z}_k^t\}_{k=0}^K} Q(\{\mathbf{Z}_k^t\}_{k=0}^K) \frac{P(\mathbf{A}^t, \{\mathbf{Z}_k^t\}_{k=0}^K)}{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)} \quad (17)$$

$$= \log \mathbb{E}_{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)} \left[\frac{P(\mathbf{A}^t, \{\mathbf{Z}_k^t\}_{k=0}^K)}{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)} \right]. \quad (18)$$

By applying Jensen's Inequality ($\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$), we establish the lower bound:

$$\log P(\mathbf{A}^t) \geq \mathbb{E}_{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)} \left[\log \frac{P(\mathbf{A}^t, \{\mathbf{Z}_k^t\}_{k=0}^K)}{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)} \right] \quad (19)$$

$$= \sum_{\{\mathbf{Z}_k^t\}_{k=0}^K} Q(\{\mathbf{Z}_k^t\}_{k=0}^K) \log \frac{P(\mathbf{A}^t, \{\mathbf{Z}_k^t\}_{k=0}^K)}{Q(\{\mathbf{Z}_k^t\}_{k=0}^K)}. \quad (20)$$

B Implementation Details

B.1 The Instantiation of MoVE

We provide details on extending our framework to the continuous-time setting, along with the implementation and architectural backbone of MoVE.

Continuous-Time Encoder Architecture. In the continuous-time setting, for a given potential interaction (u, v, t) , the encoder ϕ processes historical context of the involved nodes to compute the parameters of the posterior distribution $\boldsymbol{\mu}^t$ and $\boldsymbol{\sigma}^t$. Specifically, we collect the historical first-hop interactions for the source node u and v before timestamp t , these sequences of historical events are denoted as: $\tilde{N}_u^t = \{(u, u', t') \mid t' < t\} \cup \{(u', u, t') \mid t' < t\}$, and $\tilde{N}_v^t = \{(v, v', t') \mid t' < t\} \cup \{(v', v, t') \mid t' < t\}$, respectively. For each historical interaction (u, v, t') with $t' < t$, the concatenated feature is constructed as:

$$\mathbf{h}_{u,v}^{(l-1)} = [\mathbf{x}_u^t \mathbf{W}_N \parallel \mathbf{e}_{u,v}^t \mathbf{W}_E \parallel \tau(t - t') \mathbf{W}_T], \quad (21)$$

where $\mathbf{W}_N, \mathbf{W}_E, \mathbf{W}_T$ are trainable weight matrices corresponding to node features, edge features, and temporal encodings, respectively.

The representations $\mathbf{H}_{u,t}^{(l-1)}$ and $\mathbf{H}_{v,t}^{(l-1)}$ are formed by stacking the embeddings of all historical neighbors in $\tilde{N}_u^t$ and $\tilde{N}_v^t$, respectively, $\mathbf{H}_t^{(l-1)}$ is obtained by concatenating $\mathbf{H}_{u,t}^{(l-1)}$ and $\mathbf{H}_{v,t}^{(l-1)}$ to capture temporal dependencies both within a node's historical sequence and across its neighbors' sequences. This sequence of historical embeddings is fed into a multi-layer Transformer encoder to capture temporal dependencies and interactions among neighbors. For the l -th layer, given the input representations $\mathbf{H}_t^{(l-1)}$, the update is defined as:

$$(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{LN}(\mathbf{H}_t^{(l-1)}) (\mathbf{W}_Q^{(l)}, \mathbf{W}_K^{(l)}, \mathbf{W}_V^{(l)}) \quad (22)$$

$$\mathbf{H}_t^{(l)} = \text{MultiHeadAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) + \mathbf{H}_t^{(l-1)}, \quad (23)$$

$$\boldsymbol{\mu}^t = \text{FFN}_\mu(\text{LN}(\mathbf{H}_t^{(l)})) + \mathbf{H}_t^{(l)}, \quad \boldsymbol{\sigma}^t = \text{FFN}_\sigma(\text{LN}(\mathbf{H}_t^{(l)})) + \mathbf{H}_t^{(l)}. \quad (24)$$

where $\mathbf{W}_Q^{(l)}, \mathbf{W}_K^{(l)}, \mathbf{W}_V^{(l)}$ are trainable projection matrices, Multi-HeadAttention denotes the standard multi-head scaled dot-product attention, and FFN is a position-wise feed-forward network. We make use of the reparameterization trick for training.

Environment Estimators. The K perturbators $\{g_{\omega_k}\}_{k=1}^K$, which generate the parameters for the pseudo-environments, are implemented as simple affine transformations.

$$(\boldsymbol{\mu}_k^t, \boldsymbol{\sigma}_k^t) = g_{\omega_k}(\boldsymbol{\mu}^t, \boldsymbol{\sigma}^t), \text{ where} \quad (25)$$

$$\boldsymbol{\mu}_k^t = \mathbf{W}_\mu^{(k)} \boldsymbol{\mu}^t + \mathbf{b}_\mu^{(k)}, \quad \boldsymbol{\sigma}_k^t = \text{Softplus}(\mathbf{W}_\sigma^{(k)} \boldsymbol{\sigma}^t + \mathbf{b}_\sigma^{(k)}). \quad (26)$$

The estimators are optimized with the shift loss $\mathcal{L}_{\text{shift}}$ from Eqn. (8).

Mixture of Experts. From each of the $K + 1$ posterior distributions, we sample a latent variable $\mathbf{Z}_k^t$ using the reparameterization trick. The final node representation $\mathbf{H}^t$ is obtained by a weighted aggregation of the outputs from the expert models $\{\xi_k\}_{k=0}^K$. Each expert is implemented as a linear transformation:

$$\mathbf{H}^t = \sum_{k=0}^K \lambda_k \xi_k(\mathbf{Z}_k^t), \quad (27)$$

$$\xi_k(\mathbf{Z}_k^t) = \mathbf{W}^{(k)} \mathbf{Z}_k^t + \mathbf{b}^{(k)}. \quad (28)$$

The weights $\boldsymbol{\lambda} = [\lambda_0, \dots, \lambda_K]^T$ are determined by a gating model ψ , which operates on all latent variables to assign appropriate weights to each component. For each expert k , we compute a graph-level summary vector $\bar{\mathbf{p}}_k^t$ by applying a readout function followed by a mean operation over all nodes:

$$\bar{\mathbf{p}}_k^t = \text{Mean}(\text{Tanh}(\mathbf{W}_r \mathbf{Z}_k^t + \mathbf{b}_r)), \quad (29)$$

where $\mathbf{Z}_k^t$ denotes the latent node representations from expert k , and $\mathbf{W}_r, \mathbf{b}_r$ are the parameters of the readout layer. The summary vectors are concatenated to form a context vector $\mathbf{c}^t$ that captures the information from all pseudo-environments:

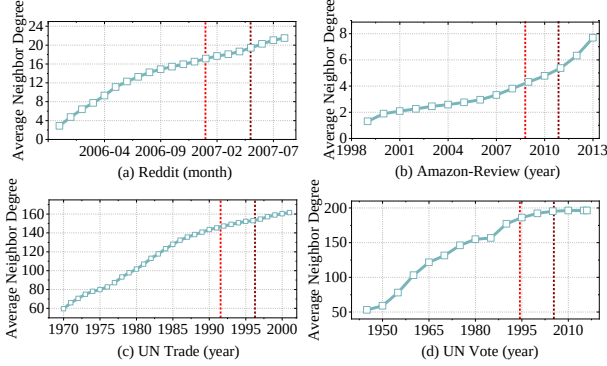
$$\mathbf{c}^t = \text{Concat}(\bar{\mathbf{p}}_0^t, \bar{\mathbf{p}}_1^t, \dots, \bar{\mathbf{p}}_K^t). \quad (30)$$

The context vector is fed into an MLP to generate a set of scores $\mathbf{s} \in \mathbb{R}^{K+1}$, which are subsequently normalized using the Softmax function:

$$\boldsymbol{\lambda} = \text{Softmax}(\text{MLP}(\mathbf{c}^t)). \quad (31)$$

Table 3: Statistics of the datasets.

Datasetset	Snapshots	Nodes	Duration	Unique Steps	Time Granularity	Graph Type
COLLAB	16	23,035	16 years	-	years	Discrete-time
YELP	24	13,095	24 months	-	months	Discrete-time
ACT	30	20,408	30 days	-	days	Discrete-time
Amazon-Review	-	236,579	14 years	974,708	years	Continuous-time
Reddit	-	74,220	21 months	886,290	months	Continuous-time
UN Trade	-	255	32 years	1,035,742	years	Continuous-time
UN Vote	-	201	72 years	2,426,279	years	Continuous-time

**Figure 8: Visualizations of the average neighbor degree.**

B.2 Datasets

We conduct a comprehensive evaluation of MoVE on link prediction tasks across seven real-world datasets, including three discrete-time and four continuous-time dynamic graphs. The detailed statistics of these datasets are reported in Table 3.

- **COLLAB** [26] is a dynamic co-authorship dataset (1990–2006) with 16 snapshots, where nodes are authors and edges denote collaborations across five research domains. We use the Data Mining domain as the shifting attribute. Node features are 32-d word2vec embeddings. The dataset is split into 10/1/5 snapshots for training, validation, and testing.

- **Yelp** [24] is a dynamic review network (2019–2020) with 24 snapshots. Nodes represent users and businesses, and edges are categorized reviews. The Pizza category is treated as the shifting attribute. Node features are 32-d word2vec embeddings, with a 15/1/8 split for training, validation, and testing.

- **ACT** [12] is a dynamic MOOC interaction dataset with 30 snapshots. Nodes are students and action targets, and edges correspond to five clustered action types, where the 5th cluster is the shifting attribute. Action features are embedded into 32-d vectors. The dataset is split into 20/2/8 snapshots for training, validation, and testing.

- **Amazon-Review** [8] is a dynamic bipartite review network derived from the Amazon Customer Reviews Dataset [20]. It covers the electronics category from 1997–2013, where nodes represent users and products, and edges denote time-stamped reviews with ratings. Following the benchmark setting, we retain only users

with at least 10 reviews. The prediction task is to determine which product a user will review next.

- **Reddit** [19] is a dynamic directed reply network (2005–2007), where nodes are users and edges represent replies between them. The task is to predict whether a user will reply to another at a given time.

- **UN Trade** [37] represents international trade flows of agricultural commodities among 181 countries over three decades, structured as a directed, weighted graph where nodes are countries, directed edges signify trade relationships, and weights indicate normalized import or export values.

- **UN Vote** [37] compiles recorded votes in the United Nations General Assembly from 1946 through 2020, representable as a dynamic network where countries are nodes and edge weights between nations quantify the number of resolutions where both countries cast an affirmative ("yes") vote.

We visualize distribution shifts in the four continuous-time datasets using the average neighbor degree (Figure 8). Dashed lines mark the temporal splits for training, validation, and testing. The datasets exhibit a clear upward trend, indicating significant distribution shifts over time.

B.3 Baselines

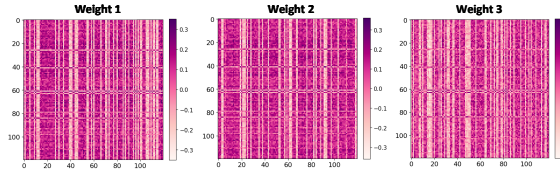
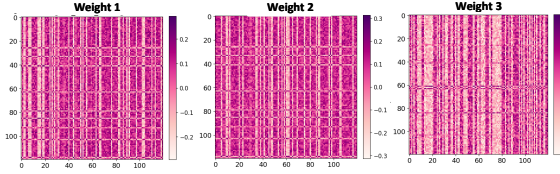
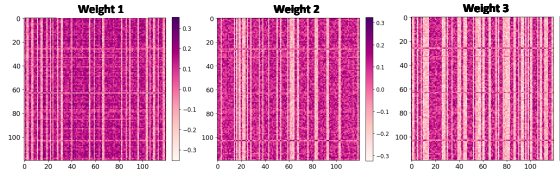
We compare MoVE with representative baselines from three categories: invariant learning, dynamic graph learning, and dynamic graph OOD generalization.

- **Invariant learning:** **IRM** [1] aims to obtain an invariant predictor across environments by minimizing empirical risk. **CaNet** [32] mitigates confounding bias by inferring pseudo-environments and guiding GNNs to learn environment-insensitive relations.

- **Dynamic graph learning:** **EGCN** [21] leverages recurrent neural networks to evolve GCN parameters across temporal snapshots. **DySAT** [24] applies self-attention to each snapshot across both structural and temporal dimensions to capture the evolution of node behaviors. **TGAT** [36] leverages self-attention on node interaction sequences to model structural and temporal dependencies. **TGN** [23] introduces a memory module to store historical information of nodes and edges, enabling efficient message passing over dynamic graphs. **GraphMixer** [4] adopts a token-mixing strategy inspired by MLP-Mixer to capture temporal dependencies. **DyGFormer** [37] encodes neighbor co-occurrence patterns and applies a patching mechanism to capture long-term dynamics. **FreeDyG** [27] introduces frequency-enhanced MLP-Mixer layer to model periodic patterns in the frequency domain.

Table 4: Experimental results on the synthetic COLLAB dataset for link prediction.

Shift	COLLAB ($\bar{p}=0.4$)	COLLAB ($\bar{p}=0.6$)	COLLAB ($\bar{p}=0.8$)
ERM	81.16 $\pm$ 0.00	81.21 $\pm$ 0.00	80.45 $\pm$ 0.00
IRM	82.42 $\pm$ 0.03	82.84 $\pm$ 0.02	78.74 $\pm$ 0.00
CaNet	80.46 $\pm$ 0.00	79.53 $\pm$ 0.00	77.61 $\pm$ 0.00
VGAE	81.72 $\pm$ 0.00	79.01 $\pm$ 0.00	81.26 $\pm$ 0.00
EGCN	80.26 $\pm$ 0.27	75.46 $\pm$ 0.13	75.35 $\pm$ 0.63
DySAT	<u>85.51 $\pm$ 0.00</u>	<u>84.98 $\pm$ 0.00</u>	<u>84.32 $\pm$ 0.00</u>
DIDA	84.59 $\pm$ 0.58	77.21 $\pm$ 0.41	72.09 $\pm$ 0.48
I-DIDA	61.59 $\pm$ 0.58	61.21 $\pm$ 0.41	60.09 $\pm$ 0.48
SILD	85.10 $\pm$ 0.00	82.17 $\pm$ 0.54	79.53 $\pm$ 0.14
EAGLE	85.19 $\pm$ 0.32	86.37 $\pm$ 0.23	80.08 $\pm$ 0.39
Ours	85.72 $\pm$ 0.00	85.09 $\pm$ 0.00	84.62 $\pm$ 0.00

**Figure 9: Visualization of environment estimator weights ($K=3$) on the COLLAB dataset.****Figure 10: Visualization of environment estimator weights ($K=3$) on the YELP dataset.****Figure 11: Visualization of environment estimator weights ($K=3$) on the ACT dataset.**

• **Dynamic graph OOD generalization:** DIDA [41] handles distribution shifts via disentangled attention and a spatio-temporal intervention to capture invariant patterns. I-DIDA [40] extends DIDA by inferring latent environments and enforcing prediction

stability across them via invariance regularization. SILD [42] decomposes graph dynamics in the spectral domain. EAGLE [38] disentangles latent environments and performs causal interventions to improve OOD generalization.

B.4 Experiment Setting Details

We expand the hyperparameter search space and carefully tune the hyperparameters as follows:

- Number of pseudo-environments (K): {2, 3, 4, 5, 6, 7, 8}
- Alignment loss weight (α): {0.0001, 0.0003, 0.0005, 0.0007, 0.0009}
- Learning rate: {0.00001, 0.00005, 0.0001}
- Scaling coefficient γ_μ : {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9}
- Scaling coefficient γ_σ : {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9}

We implement MoVE using PyTorch 2.0.0, and conduct all experiments on an NVIDIA RTX 3060 GPU with 12 GB of memory.

Algorithm 1 Overall Training Process of MoVE.

Require: Training data $\mathcal{G}^{1:T}$, max iterations T_{max} , learning rates η_θ, η_ω .

Ensure: Optimized parameters θ^* and ω^* .

- 1: Initialize main model parameters θ (encoder ϕ , experts ξ , gating ψ) and environment estimator parameters ω .
- 2: **for** iteration = 1 **to** T_{max} **do**
- 3: Sample a batch of ego-graphs (X^t, A^t) from $\mathcal{G}$.
- 4: // **1. Update Environment Estimator (ω)**
- 5: Obtain base posterior parameters μ^t, σ^t via encoder $\phi_\theta(X^t, A^t)$.
- 6: Generate perturbed parameters $\{\mu_k^t, \sigma_k^t\}_{k=1}^K$ via g_ω .
- 7: Compute extrapolation loss $\mathcal{L}_{shift}$ (Eqn. 8).
- 8: Update $\omega \leftarrow \omega - \eta_\omega \nabla_\omega \mathcal{L}_{shift}$.
- 9: // **2. Update Main Model (θ)**
- 10: Sample latent variables $\{Z_k^t\}_{k=0}^K$ from reference (q_0) and perturbed (q_k) distributions.
- 11: Compute expert gating weights λ (Eqn. 10).
- 12: Aggregate final representation H^t (Eqn. 9).
- 13: Compute ELBO loss $\mathcal{L}_1$ (Eqn. 12) and Alignment loss $\mathcal{L}_2$ (Eqn. 13).
- 14: Compute total loss $\mathcal{L}_{total} = \mathcal{L}_1 + \alpha \mathcal{L}_2$.
- 15: Update $\theta \leftarrow \theta - \eta_\theta \nabla_\theta \mathcal{L}_{total}$.
- 16: **end for**
- 17: **Return** θ^*, ω^* .

C Algorithm

The overall training process of MoVE is shown in Algorithm 1.

D Further Experiment Results

We provide additional results on distribution shifts in synthetic dynamic graphs in Table 4, along with visualizations of environment estimator weights across different branches in Figures 9, 10 and 11.